CHAPTER 5

CODING

5.1 Module Description

The system is organized into three major software modules—Data Preprocessing, Model Inference, and Visualization & Deployment—all integrated through a Dash-based web application. This architecture enables an end-to-end pipeline from raw data to interactive insights, all accessible through a user-friendly interface.

5.1.1 Data Preprocessing Module

The Data Preprocessing Module serves as the foundation for reliable and accurate model inference. Raw input data, especially in medical and neuroscience applications, is often unstructured, noisy, and heterogeneous. This module addresses such challenges through the following components:

Data Import & Format Handling:

a. Supports various data formats such as .mat, .1d, .csv, and .nii for input timeseries and anatomical atlases.
b. Automatically detects and validates file structures before processing.

Data Cleaning & Quality Control:

a. Removes or interpolates missing values.
b. Applies filters to denoise signals (e.g., band-pass filtering for brain signal timeseries).
c. Checks for data corruption, duplication, or inconsistency.

Normalization & Feature Engineering:

a. Normalizes input signals to a common scale to ensure model robustness (e.g., z-score or min-max scaling).
b. Computes statistical features (mean, variance, power) or domain-specific measures (e.g., connectivity metrics) where applicable.

Graph Construction:

a. Converts preprocessed data into graph structures using brain atlas definitions.
b. Nodes represent brain regions (ROIs); edges represent anatomical or functional connections.
c. Generates input tensors for GNN-based models, including node features (x), adjacency info (edge_index), and optional edge weights.

Batch Handling & Caching:

a. Supports processing of multiple subjects or samples in batches.
b. Implements caching mechanisms to avoid redundant computations in repeated sessions.
c. This module outputs structured data ready for direct input into the model inference pipeline.

5.1.2 Model Inference Module

The Model Inference Module handles all operations related to loading, running, and interpreting AI model predictions. It abstracts the underlying model framework (e.g., PyTorch, TensorFlow) and supports both single-sample and batch inference.

Model Initialization:

a. Dynamically loads the saved model architecture.
b. Loads pretrained weights and configures the model for inference mode (e.g., model.eval() in PyTorch).

Device Management:

a. Detects and utilizes available hardware resources (GPU if available, else CPU).
b. Ensures tensor compatibility across devices, avoiding mismatches that cause runtime errors.

Prediction Pipeline:

a. Accepts graph data (or tokenized text for NLP models) as input.
b. Performs a forward pass to compute class logits or probabilities.
c. Applies activation functions (e.g., softmax or sigmoid) and thresholding to obtain final predictions.

Confidence & Uncertainty Estimation:

a. Outputs confidence scores for each class.
b. Optionally integrates uncertainty estimation methods (e.g., Monte Carlo dropout).

Logging & Results Structuring:

a. Stores predictions, model outputs, and timestamps.
b. Logs intermediate outputs (e.g., embeddings or feature maps) for debugging or interpretability.
c. The module is designed with flexibility, supporting plug-and-play model replacement and easy integration with external model repositories.

5.1.3 Visualization Module

The Visualization Module is critical for making model outputs interpretable, especially in medical or scientific contexts. It converts predictions into interactive and informative visual formats.

3D Brain Rendering:

a. Renders anatomical brain structures using PyVista or Plotly.
b. Overlays predicted or activated regions based on model outputs, enabling spatial understanding of affected areas.

Region Highlighting:

a. Colors specific ROIs based on classification results (e.g., red for abnormal, green for normal).
b. Supports adjustable opacity, color maps, and labeling for enhanced clarity.

Time Series Animation:

a. For timeseries data, displays frame-by-frame evolution of activity over time.
b. Synchronizes brain activity with prediction confidence or class shifts.

User Interface (UI) Integration:

a. Provides UI elements such as file upload buttons, progress bars, sliders, and model selection dropdowns using Dash.
b. Enables interactive control over visualization parameters, playback speed, and ROI filters.

Export & Sharing:

a. Allows exporting visualizations as videos, images, or interactive HTML files for documentation or collaboration.
b. This module not only serves as a front-end but also aids in model interpretability and end-user accessibility.

5.2 Sample Code

```
#Pearson mat
def pearson_mat(data): #data: HO atlas of the subject
    n = data.shape[1]  # Number of features
    corr_matrix = np.zeros((n, n))  # Initialize empty matrix

    for i in range(n):
        for j in range(n):
            corr_matrix[i, j],_ = pearsonr(data[:, i], data[:, j])

    #flattened upper triangular vector
    flattened_upper_triangular=np.array(corr_matrix[np.triu_indices(N, k=1)])
    feature_mask=scipy.io.loadmat('feature_mask_pearson.mat')['feature_mask'][0]
    #feature masking through rfe
    X_features=flattened_upper_triangular[np.bool(feature_mask)]


    return X_features

#Euclidean similarity
def euclidean_similarity(vec1, vec2):
    distance = np.linalg.norm(vec1 - vec2)  # Compute Euclidean distance
    similarity = 1 / (1 + distance)  # Convert to similarity (0 to 1)
    return abs(similarity)
```

Fig 5.1: Functions for calculating Pearson Correlation and Euclidean Similarity

The Pearson Correlation is used to get linear relationships between two ROI's. This linear relationship is taken to depict the functional connectivity between different region. The end product of pearson correlation is a 110 x 110 matrix that contains functional connectivity values between all pairs of ROIs. The Pearson Correlation is given by:

$$\rho_{ij} = \frac{\sum_{t=1}^{T}(x_i(t) - \bar{x}_i)\,(x_j(t) - \bar{x}_j)}{\sqrt{\sum_{t=1}^{T}(x_i(t) - \bar{x}_i)^2}\,\sqrt{\sum_{t=1}^{T}(x_j(t) - \bar{x}_j)^2}} \tag{5.1}$$

The Euclidean similarity function is a distance measure to calculate the similarity between the feature vectors of two subjects. The Euclidean distance is taken for the vectors and reciprocated to get the similarity measure.

$$S(A,\ B) = \frac{1}{1 + d(A,\ B)} \tag{5.2}$$

```
#For categorical types
def kronecker_cat(mat):
  out_mat=np.zeros((len(mat), len(mat)))
  for i in range (len(mat)):
    for j in range (len(mat)):
      if mat[i]==mat[j] and i!=j:
        out_mat[i][j]=1
      else:
        out_mat[i][j]=0
  return out_mat

#For continuous values
def kronecker_num(mat):
  thresh=7.6
  out_mat=np.zeros((len(mat), len(mat)))
  for i in range (len(mat)):
    for j in range (len(mat)):
      if (abs(mat[i]-mat[j]))<thresh and i!=j:
        out_mat[i][j]=1
      else:
        out_mat[i][j]=0
  return out_mat
```

Fig 5.2: Kronecker function for categorical values and numerical values

The Kronecker function is used to calculate values between phenotype data of two different subjects. The Kronecker function is defined differently for categorical data versus numerical data. The results of the Kronecker function are used to construct an adjacency matrix together with the Euclidean similarity values. The phenotypic data included information such age, sex, handedness, and site. Site, sex and handedness are categorical and age is numerical.

The Kronecker function involves different formulas to handle categorical versus numerical values and therefore two different function were written to handle different types of data.

5.3 Security Considerations:

5.3.1 Data Confidentiality and Access Control

To maintain data confidentiality, it is essential that sensitive data paths—such as the dataset directory (data/cpac/...)—are not publicly exposed. Hardcoding file paths in the source code increases the risk of accidental disclosure or unauthorized access. Instead, such paths should be managed through secure environment variables or backend configuration files protected by proper access controls. Additionally, care must be taken to prevent users from uploading or referencing arbitrary files. Input values such as file_id should be strictly validated against a whitelist or a regular expression pattern (e.g., ^[A-Za-z0-9_]+$) to prevent unauthorized filesystem access. Where feasible, especially when working with health-related data such as brain imaging files, datasets (e.g., .mat or .1D files) should be encrypted at rest and decrypted securely in memory during runtime to reduce the risk of data breaches.

5.3.2 Input Validation and Sanitization

A critical part of securing the application involves robust input validation. The file_id should be sanitized to prevent directory traversal attacks that could expose system files (e.g., using patterns like ../../../etc/passwd). A strict regular expression, such as ^[A-Z]{3}_\d{7}$, can be employed to ensure inputs conform to an expected format (e.g., NYU_0050999). In addition, the phenotype CSV file (asd_files.csv) and all necessary .mat files should be safeguarded from unauthorized changes. These files should not be modifiable by users once the application is deployed, and file integrity should be regularly verified to ensure they have not been overwritten or corrupted.

5.3.3 Dependency and Code Security

All third-party dependencies—including scipy, torch, pandas, and dash—should be managed using a requirements.txt or pyproject.toml file with pinned versions to ensure stability and mitigate supply chain risks. It is also important to avoid dangerous patterns such as using eval() or exec(), which can execute arbitrary code and introduce serious vulnerabilities. Model loading operations should not rely on unsafe methods like pickle.load() from untrusted sources. Before loading any model checkpoint, such as checkpoint_epoch_Final.pth, its integrity must be verified—ideally using a secure hash (e.g., SHA-256) or digital signature—to ensure it hasn’t been tampered with.

5.3.4 Model Prediction Security

To guard against Denial-of-Service (DoS) and resource exhaustion attacks, resource-intensive prediction logic should be bounded. The use of torch.no_grad() is a good practice to reduce memory usage during inference. However, additional measures such as rate limiting API calls—using Dash middleware or Flask extensions—can prevent users from repeatedly triggering predictions in a short span of time. Furthermore, model inference should ideally run in a sandboxed or containerized environment (e.g., Docker or Kubernetes) to isolate the process from the host system. For GPU-based systems, enforce GPU quotas or utilize containers with GPU runtime isolation (e.g., NVIDIA Docker) to prevent one user's workload from monopolizing GPU resources or leaking data to others.

5.3.5 Logging and Audit Trails

Secure logging practices should be adopted to ensure no sensitive data—such as brain scan signals, subject IDs, or predictions—is written to plaintext logs or system files accessible to the public. Debugging and operational logs should include only non-sensitive metadata and be routed through a logging framework that supports role-based access control and log level filtering (e.g., logging only warnings or errors in production).

5.3.6 UI and Frontend Security

On the frontend, care should be taken to sanitize and escape any user-generated content that is rendered on the screen. This includes user inputs such as file_id, and model outputs like prediction labels or confidence values. Displaying these directly in HTML without sanitization can open the door to client-side code injection attacks. Always use Dash's built-in rendering components or escape user strings before injecting them into the DOM.

5.3.7 Network and Deployment Security

For secure deployment, the Dash app must be served over HTTPS to protect data in transit. This can be achieved by running the app behind a reverse proxy such as Nginx with SSL enabled or using a WSGI server like Gunicorn configured for SSL. Moreover, it is highly recommended to implement authentication using Flask-Login, Auth0, or similar frameworks to restrict access to the prediction interface. If multiple users are involved, role-based access control should be implemented to limit permissions according to user responsibilities (e.g., admin, clinician, viewer).